

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امراللہی بیوکی

Saeed Amrollahi Boyouki

www.Linkedin.com/in/saeed-amrollahi-boyouki

Agenda 4/30

Computation, expressions, and some fundamental concepts

- Fundamental Types I/O
- Expressions: Introduction to arithmetic operators
- Writing simple computational programs
- Declaration and definition
- References and further readings
- Q&A

120 min (incl. Q & A)



Arithmetic operators

Operator	Function	Use	
+	addition	expr + expr	Additive
-	subtraction	expr - expr	
*	multiplication	expr * expr	Multiplicative
/	division	expr / expr	
%	modulus (remainder)	expr % expr	

- Precedence: Multiplicative operators have higher precedence than additive operators.
- Association: All operators are left-to-right associative.
- Division between integers results in an integer. If the quotient contains a fraction part, it is truncated.

```
int a, b, c;  
a = a + b + c; // (a + b) + c
```

```
int a, b, c;  
b = a + b + c; // (a + b) + c  
int d = a / b * c; // (a / b) * c
```

```
int q = 21 / 6; // q = 3;  
q = 21 / 7; // q = 3;
```

Arithmetic operators cont.

- The operands of % shall have integral types. It can't be applied to floating-point types.
- An example for % operator calculation: In Gregorian calendar, a year is *leap* year if it is divisible by 4 but not by 100 except that years divisible by 400.

```
if (( year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))  
    std::cout << year << " is a leap year." << '\n';
```

- If the second operand of / or % is zero the behavior is undefined.

$$a = (a / b) * b + a \% b$$

- The if statement and && and || will be discussed later.
- Unary plus and Unary minus

Assignment operator

Operator	Function	Use
=	simple assignment	lvalue = expr
+=	add and assign	lvalue += expr
-=	subtract and assign	lvalue -= expr
*=	multiply and assign	lvalue *= expr
/=	divide and assign	lvalue /= expr
%=	modulo and assign	lvalue %= expr

Compound assignment operators

L-value op= R-value

- Notational compactness
- Association: All operators are right-to-left associative.

L-value = L-value op R-value

“Longhand” simple assignment

```
i = j = k = 0;  
i++; j--; k++;  
i += j -= k;
```

Increment and decrement operators

- Increment and decrement operators are assignment operators:
- $x++/++x$ means $x += 1$ which again means $x = x + 1$
- $x--/--x$ means $x -= 1$ which again means $x = x - 1$
- The value of $++x$, is the new (that is, incremented) value of x .
- The value of $x++$, is the old value of x .

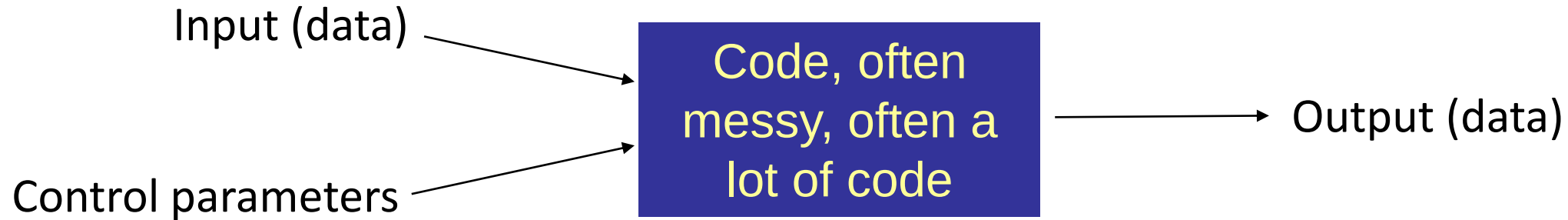
```
y = ++x; // y = (x += 1) or ++x; y = x;  
y = x++; // y = (t = x, x += 1; t) or t = x; x++; y = t;
```

```
char c = 'A';  
c++; // ok, c is 'B'  
double f = 0.0;  
f++; // ok, f is 1.0  
--3; // error: 3 is not l-value  
(1+5)++; // error: (1+5) is not l-value
```



Use prefix `++` and `--` unless you have a good reason not to.

Simple computations, simple programs



- Input: from screen, from files, from other input devices, from other programs, from other parts of a program
- Output: to screen, to files, to other output devices, to other programs, to other parts of a program
- Our job is to express computations
 - Correctly
 - Simply
 - Efficiently
- Our main tool is to break up a big computation into many little ones
 - Abstraction
 - . Provide higher-level concept that hides detail
 - “Divide and conquer”

Simple computations, simple programs

- Write a program that read a length in inches and convert it to centimeter.

```
// inch to centimeter conversion
#include <iostream>
int main()
{
    double cm_per_inch = 2.54; // number of centimeters in an inch
    std::cout << "Please enter a length in inches: (0 for exit)";
    int length; // length in inches
    std::cin >> length;
    if (length != 0) {
        std::cout << length << " in == " << cm_per_inch * length << "\n";
    }
}
```

- Write a program that read the radius of a circle and computes the area and its circumference.

```
// area and circumference
#include <iostream>
int main()
{
    double Pi = 3.14; // The pi number
    std::cout << "Please enter radius: (0 for exit)";
    int r = 0; // radius
    std::cin >> r;

    // to be continued on next page
```

Simple computations, simple programs

```
// continued from last page
double Area = Pi * r * r;
double Circ = 2 * Pi * r;
if (r != 0) {
    std::cout << "Area = " << Area << '\n';
    std::cout << "Circ = " << Circ << '\n';
}

return 0;
}
```

D eclaration and definition

D eclaration and definition

- Before a name (identifier) can be used in a C++ program, it must be declared. A *declaration* introduces names into a scope. A declaration specifies the interpretation and attributes of these names.

D eclaration and definition

- Before a name (identifier) can be used in a C++ program, it must be declared. A *declaration* introduces names into a scope. A declaration specifies the interpretation and attributes of these names.
- A declaration is a *definition* if it specifies the entity to which the declared name refers.

D eclaration and definition

- Before a name (identifier) can be used in a C++ program, it must be declared. A *declaration* introduces names into a scope. A declaration specifies the interpretation and attributes of these names.
- A declaration is a *definition* if it specifies the entity to which the declared name refers.

```
char ch; // decl. and def.
string s; // decl. and def.
int count = 1; // decl., def. and init.
struct Date { int d, m, y; }; // decl., def.
double sqrt(double); // just decl.
struct user; // just decl.
int Incr(int a); // just decl.
int Day(Date* p) { return p->d; } // def.
```

D

Declaration and definition

- Before a name (identifier) can be used in a C++ program, it must be declared. A *declaration* introduces names into a scope. A declaration specifies the interpretation and attributes of these names.
- A declaration is a *definition* if it specifies the entity to which the declared name refers.

```
char ch; // decl. and def.  
string s; // decl. and def.  
int count = 1; // decl., def. and init.  
struct Date { int d, m, y; }; // decl., def.  
double sqrt(double); // just decl.  
struct user; // just decl.  
int Incr(int a); // just decl.  
int Day(Date* p) { return p->d; } // def.
```

```
int Incr(int a) // defines Incr  
{  
    return ++a;  
}
```

D

Declaration and definition

- Before a name (identifier) can be used in a C++ program, it must be declared. A *declaration* introduces names into a scope. A declaration specifies the interpretation and attributes of these names.
- A declaration is a *definition* if it specifies the entity to which the declared name refers.

```
char ch; // decl. and def.
string s; // decl. and def.
int count = 1; // decl., def. and init.
struct Date { int d, m, y; }; // decl., def.
double sqrt(double); // just decl.
struct user; // just decl.
int Incr(int a); // just decl.
int Day(Date* p) { return p->d; } // def.
```

```
int Incr(int a) // defines Incr
{
    return ++a;
}
```

- ; is necessary, because it is a statement *terminator*.

D

Declaration and definition

- Before a name (identifier) can be used in a C++ program, it must be declared. A *declaration* introduces names into a scope. A declaration specifies the interpretation and attributes of these names.
- A declaration is a *definition* if it specifies the entity to which the declared name refers.

```
char ch; // decl. and def.
string s; // decl. and def.
int count = 1; // decl., def. and init.
struct Date { int d, m, y; }; // decl., def.
double sqrt(double); // just decl.
struct user; // just decl.
int Incr(int a); // just decl.
int Day(Date* p) { return p->d; } // def.
```

```
int Incr(int a) // defines Incr
{
    return ++a;
}
```

- ; is necessary, because it is a statement *terminator*.

- A declaration consists of four parts:
 - an optional “specifier”
 - a base type
 - a declarator
 - optional initializer

D

Declaration and definition

- Before a name (identifier) can be used in a C++ program, it must be declared. A *declaration* introduces names into a scope. A declaration specifies the interpretation and attributes of these names.
- A declaration is a *definition* if it specifies the entity to which the declared name refers.

```
char ch; // decl. and def.
string s; // decl. and def.
int count = 1; // decl., def. and init.
struct Date { int d, m, y; }; // decl., def.
double sqrt(double); // just decl.
struct user; // just decl.
int Incr(int a); // just decl.
int Day(Date* p) { return p->d; } // def.
```

```
int Incr(int a) // defines Incr
{
    return ++a;
}
```

- ; is necessary, because it is a statement *terminator*.

- A declaration consists of four parts:

- an optional “specifier”
- a base type
- a declarator
- optional initializer

```
extern double pi = 3.14;
```

D

Declaration and definition

- Before a name (identifier) can be used in a C++ program, it must be declared. A *declaration* introduces names into a scope. A declaration specifies the interpretation and attributes of these names.
- A declaration is a *definition* if it specifies the entity to which the declared name refers.

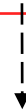
```
char ch; // decl. and def.
string s; // decl. and def.
int count = 1; // decl., def. and init.
struct Date { int d, m, y; }; // decl., def.
double sqrt(double); // just decl.
struct user; // just decl.
int Incr(int a); // just decl.
int Day(Date* p) { return p->d; } // def.
```

```
int Incr(int a) // defines Incr
{
    return ++a;
}
```

- ; is necessary, because it is a statement *terminator*.

- A declaration consists of four parts:
 - an optional “specifier”
 - a base type
 - a declarator
 - optional initializer

```
extern double pi = 3.14;
```



Base type

D

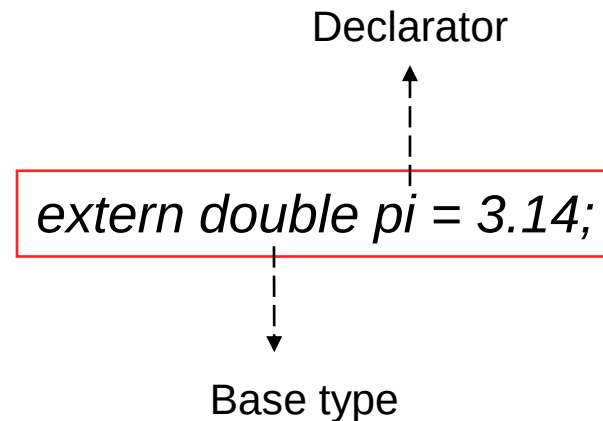
Declaration and definition

- Before a name (identifier) can be used in a C++ program, it must be declared. A *declaration* introduces names into a scope. A declaration specifies the interpretation and attributes of these names.
- A declaration is a *definition* if it specifies the entity to which the declared name refers.

```
char ch; // decl. and def.
string s; // decl. and def.
int count = 1; // decl., def. and init.
struct Date { int d, m, y; }; // decl., def.
double sqrt(double); // just decl.
struct user; // just decl.
int Incr(int a); // just decl.
int Day(Date* p) { return p->d; } // def.
```

```
int Incr(int a) // defines Incr
{
    return ++a;
}
```

- ; is necessary, because it is a statement *terminator*.



- A declaration consists of four parts:
 - an optional “specifier”
 - a base type
 - a declarator
 - optional initializer

D

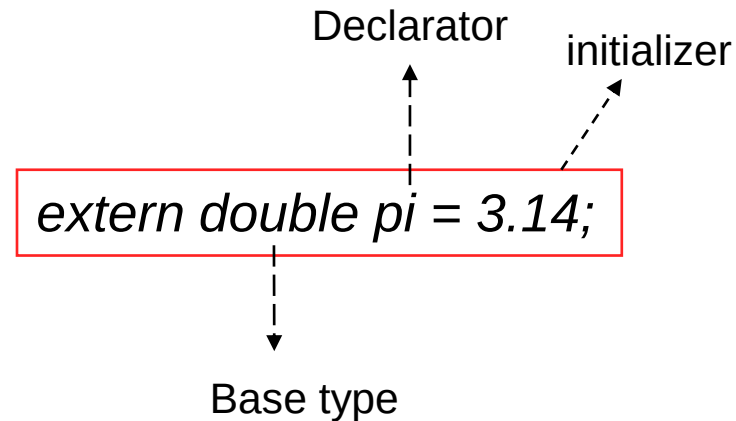
Declaration and definition

- Before a name (identifier) can be used in a C++ program, it must be declared. A *declaration* introduces names into a scope. A declaration specifies the interpretation and attributes of these names.
- A declaration is a *definition* if it specifies the entity to which the declared name refers.

```
char ch; // decl. and def.  
string s; // decl. and def.  
int count = 1; // decl., def. and init.  
struct Date { int d, m, y; }; // decl., def.  
double sqrt(double); // just decl.  
struct user; // just decl.  
int Incr(int a); // just decl.  
int Day(Date* p) { return p->d; } // def.
```

```
int Incr(int a) // defines Incr  
{  
    return ++a;  
}
```

- ; is necessary, because it is a statement *terminator*.



- A declaration consists of four parts:
 - an optional “specifier”
 - a base type
 - a declarator
 - optional initializer

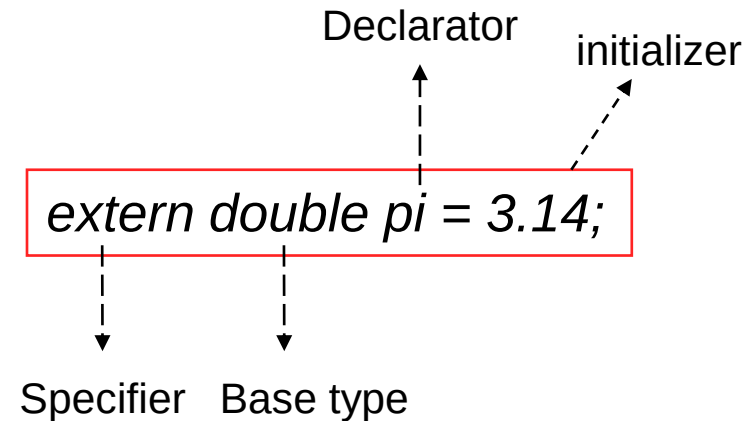
Declaration and definition

- Before a name (identifier) can be used in a C++ program, it must be declared. A *declaration* introduces names into a scope. A declaration specifies the interpretation and attributes of these names.
- A declaration is a *definition* if it specifies the entity to which the declared name refers.

```
char ch; // decl. and def.
string s; // decl. and def.
int count = 1; // decl., def. and init.
struct Date { int d, m, y; }; // decl., def.
double sqrt(double); // just decl.
struct user; // just decl.
int Incr(int a); // just decl.
int Day(Date* p) { return p->d; } // def.
```

```
int Incr(int a) // defines Incr
{
    return ++a;
}
```

- ; is necessary, because it is a statement *terminator*.



- A declaration consists of four parts:
 - an optional “specifier”
 - a base type
 - a declarator
 - optional initializer

R references and further readings

- [KM00] Andrew Koenig and Barbara Moo. Accelerated C++: Practical Programming in C++, Addison Wesley, 2000.
- [Str00] Bjarne Stroustrup. The C++ Programming Language (Special Edition). Addison Wesley. Reading Mass. USA. February 2000.
- [Str08] Bjarne Stroustrup. Programming: Principles and Practice in C++, 1st edition, Addison Wesley, December 2008.
- [Str22] Bjarne Stroustrup. A Tour of C++, 3rd edition, Addison Wesley, September 2022.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

